2026 Batch - React Architecture & Folder Structure Guide

Version: 1.0.0 **Status:** ACTIVE **React Version** 19+

This guide outlines best practices for structuring React applications for scalability, maintainability, and team collaboration. It contrasts beginner approaches with production-grade architectures and provides specific examples for Context API and Redux state management patterns.

1. The Basic Beginner Structure (And Why It Fails at Scale)

The Structure

```
src/
├── components/      # All shared components (Header, Button, UserCard, Modal...)
├── pages/           # Route pages (Home, About, Dashboard...)
├── assets/          # Images and global styles
├── App.jsx          # Routes definition
└── index.js         # Entry point

```

Why This Fails for Large Teams/Apps

While excellent for learning or small prototypes, this "Layer-Based" structure collapses as the application grows:

1. **"Drawer Clutter"**: The `components/` folder eventually holds hundreds of unrelated files (Buttons mixed with complex UserProfileCards). Finding anything becomes a search mission.
2. **High Coupling**: Business logic often leaks into UI components because there is no dedicated place for it (like `hooks` or `services`).
3. **Refactoring Nightmares**: To delete a feature (e.g., "User Profile"), you have to hunt down its files across `pages/`, `components/`, and `assets/`. You can't just delete a folder.
4. **Collaboration Friction**: Multiple developers working on different features (e.g., Auth vs. Dashboard) end up editing the same files or folders, causing merge conflicts.

---

2. Recommended Folder Structures

We recommend a **Feature-Based (Domain-Driven)** architecture. This groups files by *what they do* (Feature) rather than *what they are* (Component/Service).

Option A: Context API Structure (Small to Medium Apps)

Best for applications with moderate complexity where Redux is overkill.

```
src/
├── app/                    # Global app setup
│   └── routes/             # App routing configuration
├── assets/                 # Static assets (images, fonts)
├── config/                 # Environment variables & App Config
├── components/             # SHARED generic UI (Buttons, Inputs, Layouts)
│   ├── ui/                 # Atomic design system components
│   └── layout/             # App shells (MainLayout, AuthLayout)
├── context/                # Global contexts (ThemeContext, ToastContext)
├── features/               # ⭐ CORE: Group by Domain/Feature
│   ├── auth/               # Feature: Authentication
│   │   ├── components/     # Auth-specific UI (LoginForm)
│   │   ├── hooks/          # Logic hooks (useLogin, useAuth)
│   │   ├── context/        # Feature state (AuthContext.tsx)
│   │   └── services/       # API calls (authService.ts)
│   └── dashboard/          # Feature: Dashboard
│       ├── components/
│       └── hooks/
├── hooks/                  # Global shared hooks (useWindowSize)
├── lib/                    # 3rd party configs (Axios, Firebase)
├── styles/                 # Global CSS/SCSS
└── utils/                  # Pure helper functions (formatting)
```

```

```

Why this works:

- `features/` : Encapsulates logic. If you delete `features/auth` , you remove the entire auth feature cleanly.
- `context/` **(Global vs Local)**: Global state (Theme) lives in `src/context` , while feature state (Auth User) lives in `src/features/auth/context` . This prevents a massive "Context Hell" at the root.

Option B: Redux Toolkit Structure (Medium to Enterprise Apps)

Best for complex applications requiring strictly predictable state updates and dev tools.

```
src/
├── app/                    # App-wide settings
│   ├── store.ts            # Global Store Configuration
│   ├── rootReducer.ts      # Combine reducers
│   ├── middleware.ts       # Custom middleware (Logger, Auth)
│   └── routes.tsx          # Centralized Route definitions
├── assets/                 # Static assets imported in code
│   ├── images/
│   ├── fonts/
│   └── icons/              # SVG icons
├── config/                 # Environment config & Feature flags
│   ├── env.ts              # Type-safe env variables
│   └── constants.ts        # App-wide constants (DATE_FORMATS)
├── components/             # SHARED generic UI (No business logic)
│   ├── ui/                 # Atomic/Shadcn UI (Button, Input)
│   ├── common/             # Common composites (PageHeader, LoadingSpinner)
│   └── forms/              # Shared form inputs/controls
├── features/               # ⭐ CORE: Group by Domain/Feature
│   ├── cart/               # Feature: Shopping Cart
│   │   ├── components/     # Cart-specific UI (CartItem, CartSummary)
│   │   ├── hooks/          # Cart logic (useCartTotal)
│   │   ├── slices/         # Redux: cartSlice.ts
│   │   ├── services/       # API: cartService.ts
│   │   ├── types/          # Types: Cart.ts
│   │   └── index.ts        # Public API of the feature
│   └── auth/               # Feature: Authentication
│       ├── components/
│       ├── slices/
│       └── ...
├── hooks/                  # Global hooks (useAppDispatch, useTheme)
├── layouts/                # Page layouts (DashboardLayout, AuthLayout)
├── lib/                    # 3rd party lib wrappers (Axios, Firebase, Analytics)
│   └── axios.ts            # Axios interceptors setup
├── services/               # Global API setup (Base types, Error handling)
├── styles/                 # Global styles
│   └── globals.css         # Tailwind directives / Global resets
├── types/                  # Global TypeScript interfaces (User, AuthState)
└── utils/                  # Pure helper functions (formatDate, currency)

```

Why this works:

- `config/` **vs** `constants/` : Keeps environment variables (`config.ts`) separate from static app values (`constants.ts`).
- `lib/` **vs** `services/` : `lib` is for configuring external libraries (e.g., setting up an Axios instance), while `features/*/services` uses that instance to make actual API calls.
- `slices` **in Features**: Following the "Ducks" or "Slice" pattern, the Redux logic lives *with* the UI that uses it.
- `test/` : A dedicated place for global test configuration (like custom renderers) and mocks.
- `index.ts` **in Features**: Allows you to export only what's needed from a feature, keeping internal implementation details private (Encapsulation).

---

3. Core Principles of Scalable Architecture

Scalability

The ability of the codebase to handle growth (more features, more data) without becoming unmanageable.

- **How to achieve**:
  - **Feature-Based Folders**: Adding a new feature = adding a new folder in `features/`.
  - **Encapsulation (Barrel Files)**: Use `index.ts` in feature folders to export *only* the public components/hooks. This hides internal implementation details and prevents "spaghetti imports" across features.

Collaboration

The ability for multiple developers to work simultaneously without stepping on each other's toes.

- **How to achieve**: Modular code. Developer A works in `features/cart`, Developer B works in `features/auth`. Minimal file overlap decreases merge conflicts.

Maintainability

The ease with which code can be understood, fixed, and updated.

- **How to achieve**:
  - **Separation of Concerns**: Logic lives in Hooks/Slices, UI lives in Components.
  - **DRY (Don't Repeat Yourself)**: Extract common logic to `utils/` or custom hooks.

Reusability

Writing code once and using it everywhere.

- **How to achieve**: Moving generic components (Buttons, Cards) to `src/components/ui`. Feature-specific code stays in `features/` and isn't forced to be reusable if it doesn't need to be.

---

4. Best Practices for System Design in React

1. State Management Patterns

- **Server State (React Query/SWR)**: Use these for data fetched from APIs. Do not put API data in Redux unless absolutely necessary (caching/deduping is handled better by Query libs).
- **Client State (Redux/Context)**: Use for UI state (isSidebarOpen, themeMode) or complex cross-feature data (User Session, Shopping Cart).
- **Local State (useState)**: Use for form inputs and toggle states specific to a single component.

2. Separation of UI and Business Logic

- **Pattern**: "Hooks-based Separation".
- **Rule**: UI components should *render* data, not *fetch* or *process* it.
- **Bad**: Calling `fetch()` inside a `useEffect` directly in a component.
- **Good**: Calling `const { data } = useProducts()` custom hook. The hook handles the fetching/loading/error logic.

3. Reusable Component Design

- **Props Interface**: Define strict TypeScript interfaces for props.
- **Composition**: Use `children` prop to allow flexibility rather than passing 10 boolean flags to control content.
- **Atomic Design**: Build small atoms (Button), molecules (SearchInput), and organisms (Header).

4. Consistent Naming Conventions

- **Components**: PascalCase (`UserProfile.tsx`).
- **Hooks**: camelCase, prefix with 'use' (`useAuth.ts`).
- **Functions/Variables**: camelCase (`fetchUserData`).
- **Constants**: SCREAMING_SNAKE_CASE (`MAX_RETRY_COUNT`).
- **Types/Interfaces**: PascalCase, often prefixed/suffixed (e.g., `IUser` or `UserType`).

5. API Layer Abstraction

- **Centralized Client**: Create a dedicated API client (e.g., `lib/axios.ts`) with interceptors for auth tokens and error logging.
- **Service Layer**: Components should call `authService.login(creds)` , not `axios.post('/login')` . This allows you to swap API implementation later without breaking components.

6. Error & Loading States

- **Centralized Handling**: Use Global Error Boundaries for app crashes.
- **UI Feedback**: Always handle `isLoading` and `isError` states. Do not leave the user staring at a blank screen while data loads.
- **Skeletons**: Prefer Skeleton loaders over spinning wheels for better perceived performance.

7. Performance & Code Splitting

- **Route-Level Splitting**: Lazy load top-level pages using `React.lazy()` and `Suspense` .
- **Memoization**: Use `useMemo` for expensive calculations and `useCallback` for functions passed to memoized children.
- **Virtualization**: Use libraries like `react-window` for rendering long lists (100+ items) to maintain DOM performance.

8. Security Best Practices

- **XSS Protection**: Never use `dangerouslySetInnerHTML` unless sanitized (e.g., via DOMPurify).
- **Auth Storage**: Prefer `HttpOnly` cookies over `localStorage` for storing sensitive tokens to prevent XSS attacks.
- **Dependencies**: Regularly run `npm audit` to check for vulnerable packages.

9. Accessibility (a11y)

- **Semantic HTML**: Use `<button>` , `<nav>` , `<main>` instead of `<div>` .
- **Keyboard Nav**: Ensure all interactive elements are reachable via Tab.
- **ARIA**: Use `aria-label` only when visible text is insufficient.
- **Color Contrast**: Ensure text meets WCAG AA standards.